# NestJS – API

Step 01: Create a new project with NestCLI using following command,

**:- nest new crud-mysql-student**

Step 02: Install required dependencies like mysql2, typeorm, @nestjs/typeorm

**:- npm install @nestjs/typeorm typeorm mysql2**

Step 03: Connection can be done using two ways:-

-create a .env file and add following details in it,

**.env**

```
DB_HOST = localhost
DB_PORT = 3306
DB_USERNAME = root
 DB_PASSWORD =
DB_DATABASE = nest_db_test
```

Next Step: in app.module.ts add ConfigModule and TypeOrmModule in imports array

**app.module.ts**

```
@Module({
imports: [
ConfigModule.forRoot(),
TypeOrmModule.forRootAsync({
imports: [ConfigModule],
useFactory: (configService : ConfigService)=>({
type: 'mysql',
host: configService.get('DB_HOST'),
port: configService.get('DB_PORT'),
username: configService.get('DB_USERNAME'),
password: configService.get('DB_PASSWORD'),
database: configService.get('DB_DATABASE'),
entities: [Student],
synchronize: true
}),
inject: [ConfigService]
})
],
controllers: [AppController],
providers: [AppService],
})
export class AppModule {}
```

->Other ways to do this

-no need of creating .env file directly u can connect it using app.module.ts

**app.module.ts**

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from
'@nestjs/typeorm';
import { Student } from
'./student/student.entity';

@Module({
 imports: [`
  TypeOrmModule.forRoot({
   type: 'mysql',
   host: 'localhost',
   port: 3306,
   username: 'root',
   password: '',
   database: 'nestjs_db',
   entities: [Student],
   synchronize: true,
  }),
 ],
})
export class AppModule {}
```

# NestJS – API

Step 04: create a resource with NestCLI using following command

| |
|---|
| **:- nest generate resource student** |
| **Select:Transport layer → REST API**<br>**CRUD → Yes**<br><br>This will create 8 files and update app.module.ts |

Step 05: Modify src/student/entities/student.entity.ts file

**->Student.entitiy.ts**

```
import { Column, Entity, PrimaryGeneratedColumn } from 'typeorm';

@Entity()
export class Student {
   @PrimaryGeneratedColumn()
   StudentID: number;

   @Column()
   StudentName: string;

   @Column()
   StudentAge: number;

   @Column()
   StudentGender: string;

   @Column()
   StudentRollNo: string;

   @Column()
   StudentSemester: number;
}
```

Step 06: Modify src/student/student.service.ts file

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Student } from './entities/student.entity';

@Injectable()
export class StudentService {
  constructor(
    @InjectRepository(Student)
    private readonly studentRepo: Repository<Student>,
  ) {}
```

# NestJS – API

```
  findAll() {
    return this.studentRepo.find();
  }

  findOne(id: number) {
    return this.studentRepo.findOneBy({ StudentID: id });
  }

  create(data: any) {
    return this.studentRepo.save(data);
  }

  update(id: number, data: any) {
    return this.studentRepo.update(id, data);
  }

  delete(id: number) {
    return this.studentRepo.delete(id);
  }
}
```

Step 07: Modify src/student/student.controller.ts file

```
import { Controller, Get, Post, Body, Param, Put, Delete } from '@nestjs/common';
import { StudentService } from './student.service';
@Controller('student')
export class StudentController {
  constructor(private readonly studentService: StudentService) {}

  @Get()
  findAll() {
    return this.studentService.findAll();
  }

  @Get(':id')
  findOne(@Param('id') id: string) {
    return this.studentService.findOne(+id);
  }

  @Post()
  create(@Body() body: any) {
    return this.studentService.create(body);
  }
  @Put(':id')
  update(@Param('id') id: string, @Body() body: any) {
    return this.studentService.update(+id, body);
  }

  @Delete(':id')
  delete(@Param('id') id: string) {
    return this.studentService.delete(+id);
  }}
```

# NestJS – API